QBasic — Reference

QBasic is a Microsoft-BASIC compiler built with the self-hosted toolchain (`lex` + `yacc` + `cc`): `qbasic.l` / `qbasic.y` translate a `.bas` program to C, which `cc` lowers to a **native .NET executable** (with a Portable PDB, so runtime errors map back to the `.bas` line). Functions become `public static` methods, so a compiled program is referenceable from C#/VB.NET.

```
qbasic prog.bas -o prog.exe     # compile to a native .NET exe
prog.exe                        # run it
```

Quick Reference

```
PRINT a; b        ' ; = no gap (just spacing), , = tab zone, trailing ; = no newline
INPUT x           ' read a value
LET x = 5         ' LET optional:  x = 5
DIM a(10)         ' array 0..10;  DIM g(3,3) 2-D;  DIM n AS INTEGER
IF c THEN ... ELSEIF c2 THEN ... ELSE ... END IF      ' or single-line: IF c THEN s
FOR i = 1 TO 10 STEP 2 ... NEXT i
WHILE c ... WEND          DO ... LOOP UNTIL c          DO WHILE c ... LOOP
SELECT CASE x : CASE 1, 2 : ... : CASE IS > 5 : ... : CASE ELSE : ... : END SELECT
SUB name (a, b) ... END SUB          FUNCTION f (x) ... f = expr ... END FUNCTION
CALL name(args)           GOTO label / 100           END / STOP
' comment   (also REM)
```

Data types

Type is carried by a variable's **suffix**; unsuffixed numerics default to double.

Suffix	Type	.NET	Example
`%`	integer	Int32	`count%`
`&`	long	Int32	`total&`
`!`	single	Double	`x!`
`#`	double	Double	`pi#`
`$`	string	String	`name$`
(none)	double	Double	`x`

`DIM name(n)` creates a 0-based array (`0..n`); `DIM name(lo TO hi)` sets bounds; two dimensions are allowed (`DIM g(3,3)`). `DIM name AS INTEGER` declares a scalar's type explicitly.

Statements / Commands

`PRINT` / `INPUT`; assignment (`LET` optional); `DIM`; `IF…THEN…ELSEIF…ELSE…END IF` (and single-line `IF… THEN…`); `FOR…NEXT` (with `STEP`); `WHILE…WEND`; `DO…LOOP [WHILE|UNTIL]`; `SELECT CASE` (`CASE v`, `CASE a TO b`, `CASE IS < x`, `CASE ELSE`); `GOTO` to a name label or line number; `SUB` / `FUNCTION` + `CALL`; `CLS` / `SCREEN` / `PSET` / `LINE` / `CIRCLE` / `COLOR` (graphics); `END` / `STOP`.

Functions

`FUNCTION f (params) … f = value … END FUNCTION` returns by assigning to its own name; `SUB name (params) … END SUB` is a procedure. **Parameters are by reference** (QBasic semantics) — a SUB can modify the caller's variables. Built-in string functions: `LEFT$ RIGHT$ MID$ LEN INSTR UCASE$ LCASE$ CHR$ ASC STR$ VAL SPACE$ STRING$`; math: `ABS SQR SIN COS TAN ATN EXP LOG INT RND SGN`.

Input / Output

`PRINT` writes items; `;` separates with no extra gap, `,` tabs to the next zone, and a trailing `;` suppresses the newline. Numbers print with a leading/trailing space (the classic QBasic convention). `INPUT [prompt;] var` reads a line and parses it to the variable's type.

Graphics

`SCREEN n` opens a graphics surface (a framebuffer); `CLS`, `PSET (x,y)`, `LINE (x1,y1)-(x2,y2)[,c] [,B]`, `CIRCLE (x,y),r`, `COLOR c`. Graphics programs are viewed by running them through the windowed `gfx` host (see Activity 8).

Notes

- Compiles to a **native .NET executable**; SUB/FUNCTIONs are `public static` methods, so C#/VB.NET can reference a compiled program. A Portable PDB maps runtime errors to the original `.bas` line.
- Variables are module-global C globals; SUB/FUNCTION parameters are by reference; there are no per-SUB local variables.
- Case-insensitive keywords; numbers print with QBasic's leading-space convention.

Subset boundaries

A large, practical subset — not 100% of QuickBASIC. Not included: `GOSUB` / `RETURN`; separate-module compilation (`$INCLUDE`); `TYPE…END TYPE` records; `READ` / `DATA`; fixed-length strings; `PRINT USING`; true single vs double distinction (both are Double); event traps. Arrays use constant integer bounds.

Tutorial

Every example below was compiled with `qbasic` and run; the output shown is the real output.

1. Your first program

```
PRINT "Hello, QBasic!"
```

```
Hello, QBasic!
```

A program is a sequence of statements; no `main` is needed. Compile and run with `qbasic hello.bas -o hello.exe` then `hello.exe`.

2. Variables and data types

```
x = 3.5
y = 2
a$ = "hello"
b$ = "world"
PRINT "x + y ="; x + y
PRINT a$ + " " + b$
```

```
x + y = 5.5
hello world
```

`x` / `y` are doubles; `a$` / `b$` are strings. `+` adds numbers and concatenates strings. (Numbers carry the leading/trailing space.)

3. Flow control

```
FOR i = 1 TO 5
    PRINT i; "squared ="; i * i
NEXT i
IF x > y THEN
    PRINT "x is bigger"
ELSE
    PRINT "y is bigger"
END IF
n = 1
WHILE n ≤ 3
    PRINT "n ="; n
    n = n + 1
WEND
SELECT CASE i
  CASE 1, 2:    PRINT "small"
  CASE 6:       PRINT "it is six"
  CASE ELSE:    PRINT "other"
END SELECT
```

(from `t1.bas`, with `x=3.5, y=2`) →

```
 1 squared = 1
 2 squared = 4
 3 squared = 9
 4 squared = 16
 5 squared = 25
x is bigger
n = 1
n = 2
n = 3
it is six
```

`FOR` / `WHILE`, block `IF`, and `SELECT CASE` (with comma lists and `CASE ELSE`) all work; `FOR … STEP -1` counts down.

4. Arrays

```
DIM a(5)
FOR i = 0 TO 5
    a(i) = i * i
NEXT i
FOR i = 0 TO 5
    PRINT a(i);
NEXT i
PRINT
DIM grid(3, 3)
grid(1, 2) = 99
grid(2, 1) = 42
PRINT "grid:"; grid(1, 2); grid(2, 1)
```

```
 0  1  4  9  16  25
grid: 99  42
```

`DIM a(5)` is a 0-based array (`0..5`); two-dimensional arrays (`grid(3,3)`) index as `grid(r, c)` .

5. Subroutines and functions

```
DECLARE FUNCTION cube (n)
x = 10: y = 20
PRINT "before:"; x; y
CALL Swap(x, y)
PRINT "after:"; x; y
z = 5
CALL AddOne(z)
PRINT "addone:"; z

FUNCTION cube (n)
  cube = n * n * n
END FUNCTION
SUB Swap (a, b)
  t = a: a = b: b = t
END SUB
SUB AddOne (n)
  n = n + 1
END SUB
```

```
before: 10  20
after: 20  10
addone: 6
```

Parameters are **by reference**: `Swap` exchanges the caller's `x` and `y`, and `AddOne` increments the caller's `z`. A `FUNCTION` returns a value by assigning to its own name (`cube = …`).

6. Memory management

QBasic has no manual memory management — the .NET garbage collector owns it. The model to know:

- **Scalars and arrays** are module-level globals; an `int` / `double` is a value, a `string` is a managed `System.String` reference.
- **Arrays** (`DIM`) are fixed-size storage allocated once.
- **SUB/FUNCTION parameters are by reference**: passing a variable lets the procedure write back to it (Activity 5); passing a literal or expression makes a temporary.

There is nothing to free; reassigning a string just drops the old one for the GC.

7. Strings and a text layout

```
s$ = "QBasic"
PRINT MID$(s$, 2, 3); " "; RIGHT$(s$, 3); " "; LEFT$(s$, 1)
PRINT "instr:"; INSTR(s$, "as")
```

```
Bas sic Q
instr: 3
```

`MID$(s,2,3)` = `"Bas"`, `RIGHT$(s,3)` = `"sic"`, `LEFT$(s,1)` = `"Q"`; `INSTR` returns the 1-based position of a substring. With `LEN`, `SPACE$`, and string `+` you can build aligned columns.

8. Drawing a picture

QBasic has real graphics — `SCREEN` / `PSET` / `LINE` / `CIRCLE` / `COLOR` draw to a framebuffer:

```
SCREEN 12
CLS
COLOR 14
LINE (10, 10)-(300, 200), 12
CIRCLE (320, 240), 60, 11
SLEEP
```

This compiles and is viewed by running it through the windowed `gfx` host (graphics go to a window, not the console). For a result you can see right here in text, the same control flow draws **text art**:

```
FOR r = 1 TO 4
    s$ = ""
    FOR c = 1 TO r
        s$ = s$ + "*"
    NEXT c
    PRINT s$
NEXT r
```

```
*
**
***
****
```

9. Where to go next

A compiled QBasic program is an ordinary .NET assembly. Compile a file of `SUB` / `FUNCTION` s and call them from C#/VB.NET, or compile to a native exe and run it directly. From here, explore the graphics words through the `gfx` host, and read the generated C / Portable PDB to see how each statement lowers. Remaining gaps (`GOSUB` , `TYPE` , `READ` / `DATA`) are listed under *Subset boundaries*.